读题指南 —— 写给刚学完 C++ 基础语法的大一新生

> 这份指南不讲算法、不讲语法，只教你一件事：**把题面"翻译"成代码任务。**
很多题你明明会写，却一直 WA（Wrong Answer），八成不是代码错了，而是题面里的某个字没看到。
>
> 配套请看同目录下的《新手刷题指南》（刷题规范 + 55 题技能地图）。本指南偏**读题技巧**，那份偏**方法论与路线图**，配合使用。
本文 4.x 各节和 **5.5 的"翻车点→真实题号"对照表** 都是照 001基础语法55题 里**真实题解核过**的案例。

---

0. 这份指南怎么用

- **做每道题前**：按「读题三步法」把题念一遍，用**三句话**复述给同学听：输入是什么？输出是什么？要注意什么？
- **做每道题后**：对照「提交前 check 清单」逐项过一遍，再交。
- **WA 了先别看题解**：按「高频翻车点」逐条排查——先翻 **5.5 对照表**，找到你手头这题属于哪类坑，90% 的情况你能自己找到原因。

---

1. 一道洛谷题的「地图」——题面由哪几部分构成

拿到题，先认清它长什么样。洛谷（以及大多数 OJ）的题面固定由这几块组成：

区块	回答的问题	最容易看漏的地方
题目背景 / 题目描述	这题在讲一个什么故事，要我算什么东西	把"人话"翻译成"公式/操作"
输入格式	程序要从键盘读**哪些**数，**什么顺序**，**一行还是多行**	顺序、分隔符、数据组数
输出格式	我的程序要**输出什么**，**几行**，**什么格式**	保留几位小数、空格、换行、大小写、要不要空行
样例输入 / 样例输出	给一组例子，告诉你"你理解对了没"	用来**验证理解**，不是让你照着抄
提示 / 数据范围	n 有多大？数有多大？	决定你**选什么类型、数组开多大**

关键心态：读题不是读故事，是**找信息。**建议拿笔在题面上画：圈出所有"输入/输出的量"，在"输入格式""输出格式"下划线，把"保证/恰好/不超过/互不相同"这类词打星号。

---

2. 读题三步法（核心方法论）

每道题，按这三步走一遍：

第一步 圈输入量

问自己三个问题：

- 要读**几个**数？分别是什么？——是整数 `int`，还是小数 `double`，还是字符串 `string`？
- 它们是**同一行**读（`cin >> a >> b;`）还是**分多行**读？
- 读到什么时候结束？（固定个数？读到 `0`？读到文件尾 `EOF`？）

第二步 找输出规则

- 输出**怎么由输入算出来**？是四则运算？是分支判断？是循环累加？
- 输出**几行**？一行里**几个数**？数之间**有没有空格**？
- 结果**要不要保留小数**？保留几位？四舍五入还是截断？

第三步 核对格式（最容易丢分）

- 如果样例可以手算，先尝试自己手算一遍，确认你理解的和题目**一模一样**。
- 把题面的**输出格式**抄到草稿纸上，写代码时对着它敲，**一个字符都不差**。

> **记住**：OJ 判题是**逐字符比对**的。你输出的 `3.14` 和 `3.140`，在题目要求保留 3 位小数时，是**两个不同的答案**。
>
> 在对比代码输出和标准输出时，系统将忽略每一行结尾的空格，以及最后一行之后多余的换行符。

---

3. 数据范围在「暗示」你

题目给的数据范围不是摆设，它在告诉你**该用哪种类型、数组开多大**。养成"看到范围就反应"的习惯。

数据范围	你该怎么做	本套题里的实证
数 ≤ 约 2×10^9（`int` 上限）	用 `int` 就够；但**两个 int 相加或者相乘**可能会溢出，要斟酌要不要 `long long`	0029 P1554 用 `long long` 存计数
数可能超过 `int`	用 `long long`（上限约 9×10^{18}）	0049 B2129 分母是**两个 int 相乘**，大范围会溢出到约 4×10^{12}
有小数 / 要精度	通常用 `double`，较少用 `float`（`float` 只有 6~7 位有效数字，精度会差一点点）	0011 P5708 三边长用 `double` 读；0007/0010/0013 都是浮点
增长极快（阶乘/次方/大计数）	别等溢出了才换类型	0051 P5739：13! 已超 21 亿，`int` 必炸
$n \leqslant 10^5$ 这种数组规模	数组开 `a[n + 5]`，**多开几格**防越界（如 $n \leqslant 100$ 就开 `a[105]`）	0021 P5718 开 `a[105]`；0044 P5738 评委 ≤20 开 `scores[25]` 留缓冲
输入一行带空格的句子	`cin >> s` 读到空格就停，要**整行读**得用 `getline(cin, s)`	0033 P5015、0035 P1765 都是含空格的整行
题目是"权重/百分比"计算	浮点乘完再转 int 会**截断**，化成整数乘除更稳	0012 P3954：`(a*2+b*3+c*5)/10` 优于 `a*0.2+...`

一句话：题面给范围，是在提前告诉你"这里用 `long long`""这里数组要开大"，**别等 WA 了才想起来。**

---

4. 高频翻车点清单（你题集里的真实例子）

4.1 题面说"输出两行"，你只输出了一行 —— `endl` 的锅

洛谷 P5706 再分肥宅水
题面输出格式写着：

```
输出两行。
第一行输出一个三位小数，表示可以获得多少毫升饮料。
```

很多人只写了：

```
cout << fixed << setprecision(3) << a / b;    // 只输出了第一行
cout << b * 2;                                // 第二行和第一行连在一起了！
```

结果：两行输出黏成一行，WA。
正确：

```
cout << fixed << setprecision(3) << a / b << endl;  // 第一行，换行
cout << b * 2 << endl;                              // 第二行
```

教训：题面里凡是出现"**每行**"、"**一行一个**"、"**输出两行**"这种词，就是在提醒你要用 `<< endl;`（或 `\n`）控制换行。**数一数题面说几行，你就写几个输出、每行结尾一个 `endl`。**

4.2 数之间要有空格，少一个空格就错

洛谷 B2023 空格分隔输出
题面要求把读入的字符、整数、单精度浮点、双精度浮点**按顺序用空格分隔**输出，浮点保留 6 位小数。正确写法：

```
cout << a << " " << b << " " << c << " " << d << endl;
```

如果写成 `cout << a << b << c << d;`，所有数挤在一起，WA。

教训：题面写"空格分隔""以空格隔开""每个数之间一个空格"，就一定要在输出里**显式加 `" "`**。你可以数一下：**输出 n 个数，中间有几个空格**？通常是 `n-1` 个（最后一个数后面没有空格）。

4.3 它是"数"，但按"字符串"读更省事

洛谷 P5705 数字反转
题面：输入一个带小数点的数（如 `123.4`），把它**反过来**输出（`4.321`）。
有人想：这是小数，用 `double` 读，再数学反转——复杂、还容易丢精度。
其实它就是个字符串：

```
string s;
cin >> s;                       // 直接把 "123.4" 读进来
reverse(s.begin(), s.end());    // 倒过来
cout << s;                      // 输出 4.321
```

> **教训**：看到"数字反转""数位分离""把某个数拆开处理"，先想想**能不能当成 `string` 读**。很多时候字符串比数学运算简单一百倍。判断标准：**只要不真正做算术，只是摆弄它的每一位，就当字符串。**

4.4 浮点数的区间判断，边界最坑

> **洛谷 P5714 肥胖问题**
题面按 BMI 区间判断，输出 `Underweight` / `Normal` / `Overweight`。
三个坑叠加在一起：
1. **浮点比较有边界**：`BMI >= 18.5 && BMI < 24` 这种写法，`18.5` 这个端点算不算"正常"，取决于题面说没说"含端点"——**一定要看清"含/不含、大于/不小于"**。
2. **输出要跟题面一字不差**：`Normal` 首字母大写、别写成 `normal`，大小写错也 WA。
3. **有的分支输出的还是浮点本身**：`Overweight` 分支会输出 `BMI`，这时要看题面**要不要保留小数**。

> **教训**：分支判断题，先看题面**输出的是固定单词还是算出来的数**；再看**端点归哪个分支**；最后看**大小写**。这三个任何一个错了都 WA。

4.5 "读到 0 结束" —— 哨兵值输入（while 循环）

> **洛谷 P1427 小鱼的数字游戏**
题面：读入一串数，**以 `0` 结尾**（`0` 不算有效数据），把这串数**逆序**输出。
这种"读到某个特殊值就停"的写法是固定套路：

```
while (cin >> x) {
    if (x == 0) break;   // 读到 0 就停
    a[n++] = x;          // 把有效数存起来
}
```

> **教训**：题面里出现"**以 … 开头/结尾**""**读到 … 为止**""**遇到 0 停止**"这类描述，就是要你**边读边判断、碰到就 `break`**。注意：那个 `0` 是**终止标记**，**不能**当作有效数据存进去、更不能输出。

4.6 隐藏的"限定词"，一字之差

读题时，这些词要格外敏感（圈出来）：

- **恰好 / 正好**：不能多也不能少。
- **不超过 / 至少 / 至多**：边界能不能取到，决定用 `<=` 还是 `<`。
- **互不相同 / 各不相同**：暗示要不要**去重**、要不要开 `bool` 数组标记。
- **保证 / 一定**：题目保证某些条件成立了，你可以直接利用，不用特判。但**没写"保证"**的地方就有可能有坑。
- **按从小到大 / 从大到小**：输出顺序，别反了。
- **每一行 / 一行输出一个**：换行问题（见 4.1）。

4.7 读题时最容易忽略的"算式坑"（通读 55 题总结）

有些坑不在题面的"字"里，而在"你按题面写公式时"的类型细节里。本套题里反复出现：

1. **整数除整数**：题面写"÷""平均分"，你写了 `a/b` 而 a、b 都是 int——小数部分直接被吞。

> 实证：0007 B2011、0010 B2027、0014 B2029、0049 B2129 全靠"让一边变 `double`/`(double)` 强转"过关。

2. **浮点算完再塞回 int**：`int x = a*0.2 + ...` 是**截断**不是四舍五入，浮点误差可能让你少 1 分。

> 实证：0012 P3954 加权成绩（稳妥写法 `(a*2+b*3+c*5)/10`）；0041 P5744 `score*1.2` 赋回 int 同样截断。

3. **乘法/阶乘溢出**：题面没明说，但"两个 int 相乘""阶乘""累加计数"都可能爆 `int` 上限。

> 实证：0049 分母是两个 max 的乘积；0051 的 13! 已超 21 亿；0029 计数用 `long long`。

4. **向上取整被整数除法截断**：`ceil(a/b)` 里 a、b 是 int——取整前就已经截断了，`ceil` 白写。

> 实证：0014 B2029（`ceil(20000.0/V)`）、0020 P5707（`(s+v-1)/v` 整数版公式）。

5. **下标从 0 还是 1、行还是列**：题面坐标若从 1 给，代码还按 0 基写，越界判断全错。

> 实证：0040 P1789 坐标 1≤x,y≤n；0039 B2017 旋转后行列数互换。

> **记住**：公式类题目，动笔前先把"参与运算的每个量是什么类型、会不会溢出/截断"写在草稿上——这是 55 题里 WA 率最高的一类原因。

---

5. 样例的正确用法（不是让你抄）

样例是**用来验证理解的**，不是答案模板。正确姿势：

1. **如果能算，动手算一遍**：拿草稿纸，对照题面，把样例输入**口算/笔算**出结果，看和你预期的输出一致吗？不一致 = 你题读错了。
2. **样例过了 ≠ 能 AC**：后台还有**隐藏数据**，专门测你没考虑到的边界。
3. **自己造"边界数据"**：拿最小、最大、0、负数、`n=1` 试一遍你的程序。写代码前想清楚这些情况你的逻辑会不会崩。

> 例：看到 $n \leq 10^5$，就要想"数组开够了吗？会不会越界？"。看到能取 0（如 `n = 0`），就要想"我的循环会不会不进、答案对不对"。

5.5 附：这 55 题里的"翻车点 → 真实题号"对照表

这份表是从 001基础语法55题 全部题解里**逐个核对**出来的——每一类坑，都在你实际要做的题里真实发生过。WA 了不知道错哪时，按这张表排查：

坑的类型	会翻车的真实题目	一句话提醒
漏 `endl`/换行，多行输出黏一起	**0013 P5706**（输出两行）、0047 P5737（先输出个数再换行输出年份）	题面说几行，输出就写几个、行尾 `endl`
数之间空格漏打/多打	**0005 B2023**、0008 B2010、0044 P5738	输出 n 个数中间通常 n−1 个空格
输出是固定单词/字母，大小写错	**0017 P5714**（`Normal`）、**0032 B2124**（小写 `yes/no`）	照抄题面，注意大小写
保留小数：漏 `fixed` 或漏 `setprecision`	**0005/0007/0010/0011/0013/0020/0044/0046/0049**	`fixed << setprecision(n)` 两个都要
整数除整数 = 整除，小数被吞	**0007 B2011**、0010 B2027、0014 B2029、0049 B2129	要小数就先让一边变 `double`，或用 `(double)` 强转
浮点转 int = 截断不是四舍五入	**0012 P3954**（权重）、0041 P5744（成绩 ×1.2）	权重类计算尽量化成整数乘除
数据类型不够大，溢出	**0029 P1554**（long long）、0049 B2129（分母乘积）、0051 P5739（13! 超 int）	看到可能上亿/阶乘/乘积，直接 `long long`
向上取整用错	**0014 B2029**（`ceil(浮点)`）、0020 P5707（`(s+v-1)/v`）	别拿两个 int 相除的结果去取整
读到特殊值（如 0）结束	**0026 P1427**	结束标记**不存不进数组、不输出**
输入个数未知、读到结束	**0037 P1319**	`while(cin>>x)`
一行可能含空格，用错读法	**0033 P5015**、0035 P1765	`cin>>` 遇空格就停，要 `getline`
字符数组没给 `'\0'` 留位 / 开小了	**0031/0032/0034**	长度 L 的串开 `L+1`
打擂台初始值写死成 0	**0021 P5718**、0025 P5724、0045 P5749	初始取 `a[0]`，别写 0（全负数就错）
并列取谁：`>` 还是 `>=`	**0045 P5749**	题面要"先出现"就用严格 `>`
两两配对重复/自比	**0027 P1428**、0043 P5728	`for(j=i+1;...)` 不重不漏
去掉最高最低后除数忘减 2	**0044 P5738**	丢两头，除 `m-2` 不是 `m`
闰年判断顺序反	**0018 P5716**、0047 P5737	400 → 100 → 4，或写成 `%400==0 || (%4==0 && %100!=0)`
数组下标从 0 起，越界	0021、0027、0036~0040 一带	条件写 `i<n` 不写 `i<=n`；开 `n+5` 留余量
二维：行列搞反 / 旋转方向	**0039 B2017**、0038 P5731	先在草稿纸上推坐标公式再写码
坐标从 0 还是 1 开始	**0040 P1789**	越界判断跟着起止点走（`nx>=1&&nx<=n`）
统计种类 → 忘了用桶	**0029 P1554**、0030 P2141	数组下标当种类、值当次数
拆位直接改循环变量	**0023 P1980**	先 `int t=i` 复制一份
判断"恰好一个"写成"至少一个"	**0019 P5710**	恰好 = 异或 `(A||B)&&!(A&&B)`
递归漏终止条件 / 某分支漏 return	**0051 P5739**、0053 B2144	先写"什么时候停"，每个分支都 return
整数除法喂 `ceil`/取整失效	0014、0020	向上取整的输入必须是浮点除法结果

> 用法：WA 了 → 从表里挑怀疑的类 → 打开对应题号题解 → 看它是怎么避开的 → 回自己代码找同款问题。

6. 提交前 check 清单（一页速查卡）

交代码前，对着这个清单打勾：

类型与范围

- □ 变量类型对吗？会不会溢出？（该用 `long long` 的地方用了吗？）
- □ 数组开够了吗？（`+5` 防越界？）
- □ 涉及小数用 `double` 了吗？`float` 精度够吗？

输入

- □ 读的顺序和题面一致吗？
- □ 要循环读 / 读到 `0` / 读到 `EOF` 了吗？
- □ 整行带空格的，用 `getline` 了吗？

输出

- □ 输出**几行**？行尾都有 `endl` 吗？
- □ 数之间**空格**都对吗？末尾有没有多余空格？
- □ 保留几位小数？用了 `fixed << setprecision(n)` 吗？
- □ 大小写、单词拼写和题面**一模一样**吗？
- □ 有没有**多输出了**提示文字（如 `cout<<"答案是"`）？——**绝对不要加。**

逻辑

- □ 边界（0、最小、最大、`n=1`）测过了吗？
- □ 端点"含/不含"分对了吗？
- □ 分支**输出的是固定词还是算的值**？确认了吗？

附：类型 / 输出速查表

常用类型范围（够用就行，别背）

类型	大概范围	用在哪
`int`	$\pm 2.1\times10^{9}$	普通的整数
`long long`	$\pm 9.2\times10^{18}$	可能溢出的整数
`float`	6~7 位有效数字	小规模小数（**能不用就不用**）
`double`	15~16 位有效数字	小数、精度要求高
`char`	一个字符	单个字符
`string`	一串字符	字符串、数字反转

输出小数的固定写法（背下来）

方法一

```
#include <iomanip>              // 要加这个头文件
cout << fixed << setprecision(3) << x << endl;   // 保留 3 位小数
```

- `fixed`：固定小数点写法（不写的话 `setprecision` 数的是**有效数字**，不是小数位，容易错）。
- `setprecision(n)`：保留 **n 位小数**。
- 这两者搭配才是"保留 n 位小数"，**缺一不可**。
- 注意：不想要小数的题，**不要**加 `fixed << setprecision`，否则 `3` 会被输出成 `3.000`，同样 WA。

方法二

```
#include <cstdio>               // 要加这个头文件
printf("%.3f\n",x)                              // 保留 3 位小数（x 是 double 类型的）
printf("%.3Lf\n",x)                             // 保留 3 位小数（x 是 long double 类型的）
```

换行的写法

```
cout << x << endl;   // endl：换行 + 刷新缓冲区（刷题用这个就行）
// 或
cout << x << "\n";   // \n：只换行，更快
```

给新生的最后一句：读题慢一点，WA 就少一点。你在题面上多花 3 分钟，比你在 OJ 上多交 10 次强得多。